Natural Gas Winter Hedging Optimization (Gurobi)

Team 28: Shivani Murugapiran, Jake Deleray, James Wu, Yingshan Zhang, Jiayue Du

On October 1 a U.S. gas utility decides what share h_m of each winter month's forecast demand (Nov–Feb) to lock in at NYMEX futures prices; the rest is bought at the Henry Hub spot price. This notebook builds every parameter from the files in `data/` (plus NOAA CPC heating degree days, downloaded on first run) and solves the scenario LP with Gurobi for **Texas** and **Vermont**.

$$\min_{h,c} \sum_{s\in S} \pi_s c_s$$

$$c_s = \sum_{m\in M} \left[h_m D_m F_m + (D_{sm} - h_m D_m)\tilde{P}_{sm}\right] \quad \forall s \in S$$

$$L_m \le h_m \le U_m \quad \forall m \in M, \qquad \frac{\sum_{s\in H} \pi_s c_s}{\sum_{s\in H} \pi_s} \le C_{max}$$

where $\tilde{P}_{sm} = F_m\, P_{sm}/F_{sm}$. See the README for the full parameter table and assumptions.

How to use: edit the *Settings* cell below the setup cells, then *Run All*. The notebook should sit in the repo root next to the `data/` folder.

Setup

Imports, constants and the `Config` settings object.

In [1]:
```
# !pip install -r requirements.txt   # uncomment on first run
from __future__ import annotations

import sys
import urllib.request
from dataclasses import dataclass, field
from pathlib import Path

import numpy as np
import pandas as pd

try:
    import gurobipy as gp
    from gurobipy import GRB
except ImportError as e:
    raise ImportError("gurobipy is required:  pip install gurobipy") from e
```

```
from IPython.display import Image, display
```

In [2]:
```
ROOT = Path.cwd()  # run the notebook from the repo root
DATA_DIR = ROOT / "data"
OUT_DIR = ROOT / "outputs"

MONTHS = ["Nov", "Dec", "Jan", "Feb"]
# Calendar month and year offset (relative to the winter's start year) for each del
MONTH_CAL = {"Nov": (11, 0), "Dec": (12, 0), "Jan": (1, 1), "Feb": (2, 1)}
# On the first trading day of October, NYMEX contracts 1-4 are Nov, Dec, Jan, Feb
FUTURES_COL = {"Nov": "C1", "Dec": "C2", "Jan": "C3", "Feb": "C4"}

FIRST_WINTER = 1997  # 1997-98
LAST_WINTER = 2023   # 2023-24 (last October with futures quotes in the EIA file)

HEAT_CONTENT = 1.036  # MMBtu per Mcf (≈1,036 Btu/cf), so 1 MMcf = 1,036 MMBtu

STATE_NAMES = {"TX": "Texas", "VT": "Vermont", "NY": "New York"}
DEFAULT_COMPANY = {
    "TX": "ATMOS ENERGY CORPORATION",
    "VT": "VERMONT GAS SYSTEMS INC",
    "NY": "CONSOLIDATED EDISON NEW YORK INC",
}

EIA_CONS_URL = "https://www.eia.gov/dnav/ng/xls/NG_CONS_SUM_DCU_S{st}_M.xls"
NOAA_HDD_URL = (
    "https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/"
    "{year}/StatesCONUS.Heating.txt"
)
```

In [3]:
```
@dataclass
class Config:
    states: list[str] = field(default_factory=lambda: ["TX", "VT"])
    theta: float = 2.0                    # weight multiplier on H when g = 1
    gpr_percentile: float = 0.90          # GPR threshold percentile
    gpr_month: str = "2026-09"            # month whose GPR sets g
    force_g: int | None = None            # override g (0/1)
    high_quantile: float = 0.75           # H = winters with avg spot >= this quan
    high_basis: str = "rescaled"          # define H on rescaled spot (default) or
    cmax_factor: float | None = 1.0       # Cmax = factor * H-avg cost of the 50%
    cmax_abs: float | None = None         # absolute Cmax in $ (overrides factor)
    lower: list[float] = field(default_factory=lambda: [0.0] * 4)
    upper: list[float] = field(default_factory=lambda: [1.0] * 4)
    futures: list[float] | None = None    # F_m for Nov, Dec, Jan, Feb on Oct 1 20
    share_years: list[int] = field(default_factory=lambda: list(range(2020, 2025)))
    companies: dict[str, str] = field(default_factory=dict)
    scenario_demand: str = "fitted"       # "fitted" (regression) or "actual"
    data_dir: Path = DATA_DIR
    out_dir: Path = OUT_DIR
    hdd_csv: Path | None = None
    download: bool = True
    sensitivity: bool = True
    plots: bool = True
    verbose: bool = False
```

Settings

Edit these and *Run All*. Each one corresponds to a command-line flag of `gurobi.py`.

> **Futures strip:** EIA's NYMEX series ends Apr 5 2024, so enter the **Oct 1 2026 CME settlements** for Nov / Dec / Jan / Feb in `futures`. Left as `None`, the latest October strip in the data (Oct 2 2023) is used.

In [4]:
```
cfg = Config(
    states=["TX", "VT"],          # any of "TX", "VT", "NY"
    futures=None,                 # e.g. [3.10, 3.55, 3.90, 3.70]  ($/MMBtu, Nov De
    theta=2.0,                    # weight multiplier on high-price winters when g
    cmax_factor=1.0,              # Cmax = factor x high-price avg cost of a 50% he
    cmax_abs=None,                # absolute Cmax in $ (overrides cmax_factor)
    lower=[0.0, 0.0, 0.0, 0.0],   # L_m
    upper=[1.0, 1.0, 1.0, 1.0],   # U_m
    gpr_month="2026-09",
    force_g=None,                 # None = use GPR rule; 0 or 1 to override
    high_basis="rescaled",        # "rescaled" (default) or "raw" (Milestone 3 defi
    scenario_demand="fitted",     # "fitted" regression or "actual" consumption
    companies={},                 # e.g. {"TX": "CENTERPOINT ENERGY ENTEX"}
    hdd_csv=None,                 # offline HDD file (columns state,date,hdd)
    download=True,                # fetch NOAA HDD / EIA state consumption if missi
    sensitivity=True,
    plots=True,
    verbose=False,                # True shows the Gurobi log
)
assert all(0 <= l <= u <= 1 for l, u in zip(cfg.lower, cfg.upper)), "need 0 <= L_m
```

Helpers

In [5]:
```
def _download(url: str, dest: Path) -> Path:
    dest.parent.mkdir(parents=True, exist_ok=True)
    print(f"  downloading {url}")
    req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0 (hedging-
    with urllib.request.urlopen(req, timeout=60) as r:
        dest.write_bytes(r.read())
    return dest
```

In [6]:
```
def _read_eia_sheet(path: Path, sheet: str = "Data 1") -> pd.DataFrame:
    """Read an EIA dnav workbook sheet: row 2 = source keys, row 3 = labels, data a
    raw = pd.read_excel(path, sheet_name=sheet, header=None)
    keys = raw.iloc[1].tolist()
    df = raw.iloc[3:].copy()
    df.columns = ["Date"] + [str(k) for k in keys[1:]]
    df["Date"] = pd.to_datetime(df["Date"])
    for c in df.columns[1:]:
        df[c] = pd.to_numeric(df[c], errors="coerce")
    return df.reset_index(drop=True)
```

In [7]:
```python
def winter_label(y: int) -> str:
    return f"{y}-{str(y + 1)[-2:]}"
```

In [8]:
```python
def winter_month_date(y: int, m: str) -> tuple[int, int]:
    cal, off = MONTH_CAL[m]
    return y + off, cal
```

1. Prices

Henry Hub spot P_{sm} (monthly mean of daily prices) and NYMEX futures F_{sm} on the first trading day of October (contracts 1–4 = Nov, Dec, Jan, Feb).

In [9]:
```python
def load_spot(cfg: Config) -> pd.DataFrame:
    """P_sm: monthly average of daily Henry Hub spot prices, indexed by (winter, mo
    df = _read_eia_sheet(cfg.data_dir / "RNGWHHDd.xls")
    df = df.rename(columns={"RNGWHHD": "spot"}).dropna(subset=["spot"])
    monthly = df.groupby([df.Date.dt.year, df.Date.dt.month])["spot"].mean()
    rows = {}
    for y in range(FIRST_WINTER, LAST_WINTER + 1):
        rows[winter_label(y)] = [monthly.loc[winter_month_date(y, m)] for m in MONT
    return pd.DataFrame.from_dict(rows, orient="index", columns=MONTHS)
```

In [10]:
```python
def load_futures(cfg: Config) -> tuple[pd.DataFrame, pd.Series]:
    """F_sm: Nov-Feb futures on the first trading day of October of each winter.

    Returns (historical strips, the most recent October strip in the file)."""
    df = _read_eia_sheet(cfg.data_dir / "NG_PRI_FUT_S1_D.xls", sheet="Data 2")
    df = df.rename(columns={"RNGC1": "C1", "RNGC2": "C2", "RNGC3": "C3", "RNGC4": "
    rows, dates = {}, {}
    for y in range(FIRST_WINTER, LAST_WINTER + 1):
        sub = df[(df.Date >= f"{y}-10-01")].dropna(subset=["C1", "C2", "C3", "C4"])
        if sub.empty:
            raise ValueError(f"No October {y} futures quotes found")
        rows[winter_label(y)] = [float(sub[FUTURES_COL[m]].iloc[0]) for m in MONTHS
        dates[winter_label(y)] = sub.Date.iloc[0].date()
    fut = pd.DataFrame.from_dict(rows, orient="index", columns=MONTHS)
    fut.attrs["quote_dates"] = dates
    return fut, fut.iloc[-1]
```

2. Weather: NOAA CPC heating degree days

Daily population-weighted HDD by state (`StatesCONUS.Heating.txt`), cached in `data/hdd/` , summed to monthly HDD_{sm}.

In [11]:
```python
def _parse_cpc_file(text: str) -> pd.DataFrame:
    lines = [ln.strip().strip('"') for ln in text.splitlines() if ln.strip()]
    start = next(i for i, ln in enumerate(lines) if ln.startswith("Region|"))
    header = lines[start].split("|")
    recs = []
    for ln in lines[start + 1:]:
```

```
        parts = ln.split("|")
        if len(parts) != len(header):
            continue
        st = parts[0].strip()
        for d, v in zip(header[1:], parts[1:]):
            recs.append((st, d, v))
    out = pd.DataFrame(recs, columns=["state", "date", "hdd"])
    out["date"] = pd.to_datetime(out["date"], format="%Y%m%d")
    out["hdd"] = pd.to_numeric(out["hdd"], errors="coerce")
    out.loc[out.hdd < 0, "hdd"] = np.nan  # missing-value flags
    return out
```

In [12]:
```
def load_hdd(cfg: Config, states: list[str]) -> pd.DataFrame:
    """Daily HDD (state, date, hdd) from NOAA CPC StatesCONUS.Heating.txt, cached i
    if cfg.hdd_csv is not None:
        df = pd.read_csv(cfg.hdd_csv, parse_dates=["date"])
        return df[df.state.isin(states)]
    cache = cfg.data_dir / "hdd"
    frames = []
    for year in range(FIRST_WINTER, LAST_WINTER + 2):
        f = cache / f"StatesCONUS.Heating.{year}.txt"
        if not f.exists():
            if not cfg.download:
                raise FileNotFoundError(f"{f} missing and --no-download set")
            _download(NOAA_HDD_URL.format(year=year), f)
        frames.append(_parse_cpc_file(f.read_text()))
    df = pd.concat(frames, ignore_index=True)
    return df[df.state.isin(states)]
```

In [13]:
```
def monthly_hdd(hdd: pd.DataFrame, st: str) -> pd.DataFrame:
    """HDD_sm: sum of daily HDD within each delivery month of each winter."""
    d = hdd[hdd.state == st]
    tot = d.groupby([d.date.dt.year, d.date.dt.month])["hdd"].sum(min_count=1)
    rows = {}
    for y in range(FIRST_WINTER, LAST_WINTER + 1):
        rows[winter_label(y)] = [tot.get(winter_month_date(y, m), np.nan) for m in
    out = pd.DataFrame.from_dict(rows, orient="index", columns=MONTHS)
    if out.isna().any().any():
        missing = out[out.isna().any(axis=1)].index.tolist()
        raise ValueError(f"Missing HDD for {st} in winters {missing}")
    return out
```

3. Demand

State residential + commercial consumption (MMcf → MMBtu at 1,036 MMBtu/MMcf), the utility's share of state deliveries from EIA-176, and the monthly regression $D_{state,sm} = a_m + b_m\, HDD_{sm}$.

In [14]:
```
def load_state_consumption(cfg: Config, st: str) -> pd.DataFrame:
    """Residential + commercial consumption for (winter, month), in MMBtu."""
    f = cfg.data_dir / f"NG_CONS_SUM_DCU_S{st}_M.xls"
    if not f.exists():
        if not cfg.download:
```

```
            raise FileNotFoundError(f"{f} missing and --no-download set")
        _download(EIA_CONS_URL.format(st=st), f)
    df = _read_eia_sheet(f)
    res = next(c for c in df.columns if c.startswith("N3010"))
    com = next(c for c in df.columns if c.startswith("N3020"))
    df["rc_mmcf"] = df[res] + df[com]
    tot = df.set_index([df.Date.dt.year, df.Date.dt.month])["rc_mmcf"]
    rows = {}
    for y in range(FIRST_WINTER, LAST_WINTER + 1):
        rows[winter_label(y)] = [tot.loc[winter_month_date(y, m)] * 1_000 * HEAT_CO
                                 for m in MONTHS]  # MMcf -> Mcf -> MMBtu
    return pd.DataFrame.from_dict(rows, orient="index", columns=MONTHS)
```

In [15]:
```
def utility_share(cfg: Config, st: str) -> tuple[str, float]:
    """Utility's share of state residential + commercial deliveries (EIA-176)."""
    df = pd.read_excel(cfg.data_dir / "eia176_company_deliveries.xlsx")
    df.columns = [int(c) if isinstance(c, (int, float)) else c for c in df.columns]
    df["Company"] = df["Company"].astype(str).str.strip()
    company = cfg.companies.get(st, DEFAULT_COMPANY.get(st))
    sub = df[(df.Area == STATE_NAMES[st]) & df.Item.isin(["Residential Volume", "Co
    years = [y for y in cfg.share_years if y in sub.columns]
    by_co = sub.groupby("Company")[years].sum(min_count=1)
    total = by_co.loc["Total of All Companies"]
    if company not in by_co.index:
        # fall back to the largest LDC in the state
        company = by_co.drop("Total of All Companies").mean(axis=1).idxmax()
    share = float((by_co.loc[company] / total).mean())
    return company, share
```

In [16]:
```
@dataclass
class DemandFit:
    a: pd.Series
    b: pd.Series
    r2: pd.Series
    hdd_normal: pd.Series
```

In [17]:
```
def fit_demand(state_mmbtu: pd.DataFrame, hdd: pd.DataFrame) -> DemandFit:
    """OLS per month: StateDemand_sm = a_m + b_m * HDD_sm."""
    a, b, r2 = {}, {}, {}
    for m in MONTHS:
        x, y = hdd[m].values.astype(float), state_mmbtu[m].values.astype(float)
        bm, am = np.polyfit(x, y, 1)
        resid = y - (am + bm * x)
        a[m], b[m] = am, bm
        r2[m] = 1 - resid.var() / y.var()
    return DemandFit(pd.Series(a), pd.Series(b), pd.Series(r2), hdd.mean())
```

4. Geopolitical risk indicator

$g = 1$ if the GPR value for `gpr_month` is at or above the 90th percentile of monthly GPR since 1985.

In [18]:
```python
def geopolitical_indicator(cfg: Config) -> tuple[int, float, float]:
    g = pd.read_excel(cfg.data_dir / "data_gpr_export.xls", usecols=["month", "GPR"
    g["month"] = pd.to_datetime(g["month"])
    g = g.dropna(subset=["GPR"])
    hist = g[g.month >= "1985-01-01"]["GPR"]
    thresh = float(hist.quantile(cfg.gpr_percentile))
    cur = g[g.month.dt.strftime("%Y-%m") == cfg.gpr_month]["GPR"]
    if cur.empty:
        raise ValueError(f"No GPR value for {cfg.gpr_month}")
    value = float(cur.iloc[0])
    flag = int(value >= thresh) if cfg.force_g is None else int(cfg.force_g)
    return flag, value, thresh
```

5. Assemble the model inputs for one state

Builds $F_m, \tilde{P}_{sm}, D_m, D_{sm}, H$ and π_s.

In [19]:
```python
@dataclass
class Instance:
    state: str
    company: str
    share: float
    F: pd.Series            # F_m
    F_hist: pd.DataFrame    # F_sm
    P: pd.DataFrame         # P_sm
    Ptilde: pd.DataFrame    # rescaled spot
    D: pd.Series            # D_m
    Ds: pd.DataFrame        # D_sm
    HDD: pd.DataFrame
    fit: DemandFit
    H: list[str]
    pi: pd.Series
    g: int
    gpr_value: float
    gpr_thresh: float
```

In [20]:
```python
def build_instance(cfg: Config, st: str, spot, fut_hist, F_now, hdd_daily, gpr) ->
    company, share = utility_share(cfg, st)
    state_d = load_state_consumption(cfg, st)
    HDD = monthly_hdd(hdd_daily, st)
    fit = fit_demand(state_d, HDD)

    D = share * (fit.a + fit.b * fit.hdd_normal)
    if cfg.scenario_demand == "fitted":
        Ds = share * (HDD * fit.b + fit.a)
    else:
        Ds = share * state_d
    Ds = Ds.clip(lower=0)

    Ptilde = (spot / fut_hist) * F_now

    basis = spot if cfg.high_basis == "raw" else Ptilde
    avg = basis.mean(axis=1)
```

```
    H = avg[avg >= avg.quantile(cfg.high_quantile)].index.tolist()

    g, gv, gt = gpr
    pi = pd.Series(1.0, index=spot.index)
    if g == 1:
        pi[H] *= cfg.theta
    pi /= pi.sum()

    return Instance(st, company, share, F_now, fut_hist, spot, Ptilde, D, Ds, HDD,
```

6. Cost evaluation and the Gurobi model

`solve()` is the LP: variables h_m, c_s, the scenario cost constraints, hedge bounds, and the high-price cost limit.

In [21]:
```
def scenario_costs(inst: Instance, h: pd.Series) -> pd.Series:
    """c_s for a fixed hedge vector h (used for benchmarks and Cmax calibration)."""
    hedged = (h * inst.D * inst.F).sum()
    spot_part = ((inst.Ds - h * inst.D) * inst.Ptilde).sum(axis=1)
    return hedged + spot_part
```

In [22]:
```
def high_avg(inst: Instance, c: pd.Series) -> float:
    w = inst.pi[inst.H]
    return float((w * c[inst.H]).sum() / w.sum())
```

In [23]:
```
def resolve_cmax(cfg: Config, inst: Instance) -> float | None:
    if cfg.cmax_abs is not None:
        return cfg.cmax_abs
    return cmax_from_factor(inst, cfg.cmax_factor)
```

In [24]:
```
def cmax_from_factor(inst: Instance, factor: float | None) -> float | None:
    """Cmax = factor x (high-price average cost of hedging 50% every month)."""
    if factor is None:
        return None
    ref = high_avg(inst, scenario_costs(inst, pd.Series(0.5, index=MONTHS)))
    return factor * ref
```

In [25]:
```
@dataclass
class Solution:
    status: str
    h: pd.Series | None = None
    c: pd.Series | None = None
    expected: float | None = None
    high_avg: float | None = None
    cmax: float | None = None
    cmax_dual: float | None = None
```

In [26]:
```
def solve(cfg: Config, inst: Instance, cmax: float | None) -> Solution:
    env = gp.Env(empty=True)
    env.setParam("OutputFlag", 1 if cfg.verbose else 0)
    env.start()
    mdl = gp.Model(f"hedge_{inst.state}", env=env)
```

```
    S = list(inst.pi.index)
    h = mdl.addVars(MONTHS, lb={m: cfg.lower[i] for i, m in enumerate(MONTHS)},
                    ub={m: cfg.upper[i] for i, m in enumerate(MONTHS)}, name="h")
    c = mdl.addVars(S, lb=-GRB.INFINITY, name="c")

    # (1) scenario cost definition
    for s in S:
        mdl.addConstr(
            c[s] == gp.quicksum(
                h[m] * inst.D[m] * inst.F[m]
                + (inst.Ds.at[s, m] - h[m] * inst.D[m]) * inst.Ptilde.at[s, m]
                for m in MONTHS),
            name=f"cost[{s}]")

    # (2) hedge-share bounds are variable bounds (L_m <= h_m <= U_m)

    # (3) high-price scenario cost limit
    risk = None
    if cmax is not None:
        wH = float(inst.pi[inst.H].sum())
        risk = mdl.addConstr(
            gp.quicksum(inst.pi[s] * c[s] for s in inst.H) <= cmax * wH, name="high

    mdl.setObjective(gp.quicksum(inst.pi[s] * c[s] for s in S), GRB.MINIMIZE)
    mdl.optimize()

    if mdl.Status != GRB.OPTIMAL:
        status = {GRB.INFEASIBLE: "infeasible", GRB.INF_OR_UNBD: "infeasible"}.get(
            mdl.Status, f"status_{mdl.Status}")
        mdl.dispose(); env.dispose()
        return Solution(status, cmax=cmax)

    hv = pd.Series({m: h[m].X for m in MONTHS})
    cv = pd.Series({s: c[s].X for s in S})
    dual = None
    if risk is not None:
        # $ of expected cost saved per extra $ of allowed high-price average cost
        dual = -risk.Pi * float(inst.pi[inst.H].sum())
    sol = Solution("optimal", hv, cv, mdl.ObjVal, high_avg(inst, cv), cmax, dual)
    mdl.dispose(); env.dispose()
    return sol
```

7. Reporting, benchmarks and sensitivity

Strategy comparison (0–100% and optimal hedges), the θ × C_{max} sensitivity grid, plots and CSV outputs.

In [27]:
```
def strategy_table(inst: Instance, sol: Solution) -> pd.DataFrame:
    strategies = {f"{int(k * 100)}% hedge": pd.Series(k, index=MONTHS) for k in (0,
    if sol.h is not None:
        strategies["Optimal"] = sol.h
    total_vol = inst.D.sum()
    rows = []
```

```
    for name, h in strategies.items():
        c = scenario_costs(inst, h)
        rows.append({
            "strategy": name,
            **{f"h_{m}": round(h[m], 4) for m in MONTHS},
            "expected_cost": (inst.pi * c).sum(),
            "high_price_avg_cost": high_avg(inst, c),
            "worst_winter_cost": c.max(),
            "worst_winter": c.idxmax(),
            "std_cost": np.sqrt((inst.pi * (c - (inst.pi * c).sum()) ** 2).sum()),
            "expected_$per_MMBtu": (inst.pi * c).sum() / total_vol,
        })
    return pd.DataFrame(rows).set_index("strategy")
```

In [28]:
```
def sensitivity(cfg: Config, inst: Instance) -> dict[str, pd.DataFrame]:
    """Re-solve over a grid of theta (H weight, as if g = 1; theta = 1 is the g = 0
    and Cmax factors."""
    thetas = [1.0, 1.5, 2.0, 3.0, 4.0]
    factors = [0.90, 0.94, 0.97, 1.00, 1.03, 1.06, 1.10, None]
    exp_tab, hedge_tab = {}, {}
    for th in thetas:
        pi = pd.Series(1.0, index=inst.pi.index)
        pi[inst.H] *= th
        pi /= pi.sum()
        sub = Instance(**{**inst.__dict__, "pi": pi})
        exp_row, h_row = {}, {}
        for fct in factors:
            col = "no limit" if fct is None else f"{fct:.2f}x"
            cmax = cmax_from_factor(sub, fct)
            s = solve(cfg, sub, cmax)
            exp_row[col] = s.expected if s.status == "optimal" else np.nan
            h_row[col] = "/".join(f"{v:.2f}" for v in s.h) if s.h is not None else
        exp_tab[f"theta={th}"] = exp_row
        hedge_tab[f"theta={th}"] = h_row
    return {
        "sensitivity_expected_cost": pd.DataFrame.from_dict(exp_tab, orient="index"
        "sensitivity_hedge_shares": pd.DataFrame.from_dict(hedge_tab, orient="index
    }
```

In [29]:
```
def make_plots(out: Path, inst: Instance, strat: pd.DataFrame, sens: dict | None):
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        return
    fig, ax = plt.subplots(figsize=(7, 5))
    for name, r in strat.iterrows():
        ax.scatter(r.expected_cost / 1e6, r.high_price_avg_cost / 1e6, s=60,
                   marker="*" if name == "Optimal" else "o", zorder=3)
        ax.annotate(name, (r.expected_cost / 1e6, r.high_price_avg_cost / 1e6),
                    textcoords="offset points", xytext=(6, 4), fontsize=9)
    ax.set_xlabel("Expected winter cost ($M)")
    ax.set_ylabel("Avg cost in high-price winters H ($M)")
    ax.set_title(f"{inst.state}: cost vs. high-price exposure")
```

```
    ax.grid(alpha=.3)
    fig.tight_layout(); fig.savefig(out / "strategy_frontier.png", dpi=150); plt.cl

    if sens:
        t = sens["sensitivity_expected_cost"] / 1e6
        fig, ax = plt.subplots(figsize=(9, 4))
        im = ax.imshow(t.values, cmap="viridis_r", aspect="auto")
        ax.set_xticks(range(t.shape[1]), t.columns, rotation=30)
        ax.set_yticks(range(t.shape[0]), t.index)
        for i in range(t.shape[0]):
            for j in range(t.shape[1]):
                v = t.values[i, j]
                ax.text(j, i, "infeas." if np.isnan(v) else f"{v:,.1f}", ha="center
                        fontsize=8, color="white")
        ax.set_xlabel("Cmax (multiple of 50%-hedge high-price cost)")
        ax.set_title(f"{inst.state}: expected cost ($M) by theta and Cmax")
        fig.colorbar(im, ax=ax, shrink=.8)
        fig.tight_layout(); fig.savefig(out / "sensitivity_heatmap.png", dpi=150);
```

In [30]:

```
def report(cfg: Config, inst: Instance, sol: Solution) -> None:
    out = cfg.out_dir / inst.state
    out.mkdir(parents=True, exist_ok=True)

    print(f"\n{'=' * 72}\n{STATE_NAMES[inst.state]} ({inst.state})\n{'=' * 72}")
    print(f"Utility: {inst.company}  (share of state R+C deliveries = {inst.share:.
    print("Demand regression  D_state = a_m + b_m * HDD_m")
    print(pd.DataFrame({"a_m (MMBtu)": inst.fit.a, "b_m (MMBtu/HDD)": inst.fit.b,
                        "R^2": inst.fit.r2, "normal HDD": inst.fit.hdd_normal}).rou
    print(f"\nForecast utility demand D_m (MMBtu):\n{inst.D.round(0).to_string()}")
    print(f"\nHigh-price winters H ({len(inst.H)}): {', '.join(inst.H)}")
    print(f"GPR {cfg.gpr_month} = {inst.gpr_value:.1f}, threshold = {inst.gpr_thres
          f"-> g = {inst.g}" + (f", H weights x{cfg.theta}" if inst.g else ""))
    if sol.status != "optimal":
        print(f"\nModel status: {sol.status.upper()} (Cmax = {sol.cmax:,.0f}). "
              "Loosen --cmax-factor or the hedge bounds.")
        return
    print(f"\nOptimal hedge shares: " + ", ".join(f"{m} {sol.h[m]:.1%}" for m in MO
    print(f"Expected cost      : ${sol.expected:,.0f}")
    print(f"High-price avg cost: ${sol.high_avg:,.0f}"
          + (f"  (Cmax = ${sol.cmax:,.0f})" if sol.cmax else ""))
    if sol.cmax_dual is not None:
        print(f"Shadow price of Cmax: {sol.cmax_dual:.4f} $ expected / $ of Cmax")

    strat = strategy_table(inst, sol)
    print("\nStrategy comparison:")
    print(strat.drop(columns=[f"h_{m}" for m in MONTHS]).to_string(
        float_format=lambda v: f"{v:,.2f}" if abs(v) < 100 else f"{v:,.0f}"))

    # save parameters and results
    pd.DataFrame({"F_m": inst.F, "D_m": inst.D, "L_m": cfg.lower, "U_m": cfg.upper}
        out / "parameters_month.csv")
    pd.concat({"F_sm": inst.F_hist, "P_sm": inst.P, "Ptilde_sm": inst.Ptilde,
               "D_sm": inst.Ds, "HDD_sm": inst.HDD}, axis=1).assign(
        pi_s=inst.pi, in_H=inst.pi.index.isin(inst.H), c_s_optimal=sol.c).to_csv(
        out / "parameters_scenario.csv")
    pd.DataFrame({"h_m": sol.h, "hedged_MMBtu": sol.h * inst.D}).to_csv(out / "opti
```

```
    strat.to_csv(out / "strategy_comparison.csv")

    sens = None
    if cfg.sensitivity:
        sens = sensitivity(cfg, inst)
        for k, v in sens.items():
            v.to_csv(out / f"{k}.csv")
        print("\nSensitivity - hedge shares Nov/Dec/Jan/Feb (rows theta, cols Cmax
        print(sens["sensitivity_hedge_shares"].to_string())
    if cfg.plots:
        make_plots(out, inst, strat, sens)
    print(f"\nResults written to {out}")
```

8. Load shared data

In [31]:

```
spot = load_spot(cfg)
fut_hist, latest = load_futures(cfg)
if cfg.futures is not None:
    F_now = pd.Series(cfg.futures, index=MONTHS)
else:
    F_now = latest.copy()
    print(f"NOTE: futures not set; using the October strip from "
          f"{fut_hist.attrs['quote_dates'][fut_hist.index[-1]]}. Enter the Oct 1 20
print("F_m:", F_now.round(3).to_dict())

hdd = load_hdd(cfg, cfg.states)
gpr = geopolitical_indicator(cfg)
print(f"GPR {cfg.gpr_month} = {gpr[1]:.1f}, 90th pct threshold = {gpr[2]:.1f} -> g
```

```
NOTE: futures not set; using the October strip from 2023-10-02. Enter the Oct 1 2026
settlements in cfg.futures.
F_m: {'Nov': 2.84, 'Dec': 3.233, 'Jan': 3.492, 'Feb': 3.427}
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/1
997/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/1
998/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/1
999/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
000/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
001/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
002/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
003/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
004/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
005/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
006/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
007/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
008/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
009/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
010/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
011/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
012/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
013/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
014/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
015/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
016/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
017/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
018/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
019/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
020/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
021/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
022/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
```

```
023/StatesCONUS.Heating.txt
  downloading https://ftp.cpc.ncep.noaa.gov/htdocs/degree_days/weighted/daily_data/2
024/StatesCONUS.Heating.txt
GPR 2026-09 = 155.9, 90th pct threshold = 148.1 -> g = 1
```

In [32]:
```
# Historical spot / futures ratio by winter: >1 means spot came in above the Octobe
(spot / fut_hist).round(2).style.background_gradient(cmap="RdYlGn_r", axis=None)
```

Out[32]:

	Nov	Dec	Jan	Feb
1997-98	0.960000	0.740000	0.660000	0.790000
1998-99	1.050000	0.710000	0.710000	0.670000
1999-00	0.850000	0.790000	0.810000	0.940000
2000-01	1.030000	1.640000	1.510000	1.090000
2001-02	1.060000	0.880000	0.820000	0.820000
2002-03	0.990000	1.120000	1.250000	1.830000
2003-04	0.960000	1.240000	1.190000	1.050000
2004-05	0.910000	0.870000	0.770000	0.780000
2005-06	0.740000	0.900000	0.580000	0.520000
2006-07	1.310000	0.910000	0.830000	1.000000
2007-08	1.010000	0.910000	0.980000	1.040000
2008-09	0.860000	0.730000	0.640000	0.550000
2009-10	0.820000	1.000000	1.030000	0.930000
2010-11	0.980000	1.040000	1.050000	0.960000
2011-12	0.900000	0.810000	0.650000	0.610000
2012-13	1.020000	0.890000	0.860000	0.850000
2013-14	1.010000	1.130000	1.220000	1.550000
2014-15	1.020000	0.850000	0.720000	0.690000
2015-16	0.860000	0.730000	0.830000	0.720000
2016-17	0.870000	1.130000	1.000000	0.860000
2017-18	1.030000	0.910000	1.200000	0.820000
2018-19	1.320000	1.270000	0.960000	0.850000
2019-20	1.160000	0.900000	0.770000	0.740000
2020-21	1.030000	0.840000	0.850000	1.690000
2021-22	0.900000	0.650000	0.750000	0.820000
2022-23	0.840000	0.810000	0.470000	0.350000
2023-24	0.950000	0.780000	0.910000	0.500000

9. Solve the model for each state

In [33]:
```
results = {}
for st in cfg.states:
```

```
    inst = build_instance(cfg, st, spot, fut_hist, F_now, hdd, gpr)
    sol = solve(cfg, inst, resolve_cmax(cfg, inst))
    report(cfg, inst, sol)
    results[st] = (inst, sol)
```

```
  downloading https://www.eia.gov/dnav/ng/xls/NG_CONS_SUM_DCU_STX_M.xls

======================================================================
Texas (TX)
======================================================================
Utility: ATMOS ENERGY CORPORATION  (share of state R+C deliveries = 40.7%)
Demand regression  D_state = a_m + b_m * HDD_m
      a_m (MMBtu)  b_m (MMBtu/HDD)    R^2  normal HDD
Nov  1.595830e+07        69673.894  0.472     223.778
Dec  3.124408e+07        52867.219  0.401     437.407
Jan  3.086695e+07        84518.697  0.638     470.963
Feb  2.938492e+07        86440.315  0.679     350.000

Forecast utility demand D_m (MMBtu):
Nov    12828729.0
Dec    22107291.0
Jan    28736613.0
Feb    24250350.0

High-price winters H (7): 2000-01, 2002-03, 2003-04, 2010-11, 2013-14, 2018-19, 2020
-21
GPR 2026-09 = 155.9, threshold = 148.1 -> g = 1, H weights x2.0

Optimal hedge shares: Nov 0.0%, Dec 100.0%, Jan 0.0%, Feb 57.5%
Expected cost      : $293,176,912
High-price avg cost: $344,607,101  (Cmax = $344,607,101)
Shadow price of Cmax: 0.0967 $ expected / $ of Cmax

Strategy comparison:
            expected_cost  high_price_avg_cost  worst_winter_cost worst_winter   std
_cost  expected_$per_MMBtu
strategy
0% hedge      290,710,616          371,422,629        450,096,609      2000-01 82,14
3,180                 3.31
25% hedge     293,053,356          358,014,865        423,732,355      2000-01 66,58
8,096                 3.33
50% hedge     295,396,097          344,607,101        397,368,101      2000-01 51,28
7,777                 3.36
75% hedge     297,738,837          331,199,338        371,003,847      2000-01 36,56
3,462                 3.39
100% hedge    300,081,578          317,791,574        344,639,593      2000-01 23,52
2,895                 3.41
Optimal       293,176,912          344,607,101        400,399,797      2000-01 55,00
1,775                 3.33

Sensitivity - hedge shares Nov/Dec/Jan/Feb (rows theta, cols Cmax factor):
                0.90x               0.94x               0.97x               1.00x
1.03x               1.06x               1.10x             no limit
theta=1.0  infeasible  1.00/1.00/0.59/1.00  0.09/1.00/0.00/1.00  0.00/1.00/0.00/0.57
0.00/1.00/0.00/0.14  0.00/0.47/0.00/0.00  0.00/0.00/0.00/0.00  0.00/0.00/0.00/0.00
theta=1.5  infeasible  1.00/1.00/0.59/1.00  0.09/1.00/0.00/1.00  0.00/1.00/0.00/0.57
0.00/1.00/0.00/0.14  0.00/0.47/0.00/0.00  0.00/0.00/0.00/0.00  0.00/0.00/0.00/0.00
theta=2.0  infeasible  1.00/1.00/0.59/1.00  0.09/1.00/0.00/1.00  0.00/1.00/0.00/0.57
0.00/1.00/0.00/0.14  0.00/0.47/0.00/0.00  0.00/0.00/0.00/0.00  0.00/0.00/0.00/0.00
theta=3.0  infeasible  1.00/1.00/0.59/1.00  1.00/1.00/0.00/1.00  1.00/1.00/0.00/1.00
1.00/1.00/0.00/1.00  1.00/1.00/0.00/1.00  1.00/1.00/0.00/1.00  1.00/1.00/0.00/1.00
```

```
theta=4.0  infeasible  1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00
1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00

Results written to C:\Users\Shivani Murugapiran\natural-gas-hedging-optimization\out
puts\TX
  downloading https://www.eia.gov/dnav/ng/xls/NG_CONS_SUM_DCU_SVT_M.xls

======================================================================
Vermont (VT)
======================================================================
Utility: VERMONT GAS SYSTEMS INC  (share of state R+C deliveries = 100.0%)
Demand regression  D_state = a_m + b_m * HDD_m
     a_m (MMBtu)  b_m (MMBtu/HDD)    R^2  normal HDD
Nov  -625925.022         1594.275  0.219     841.444
Dec  1116246.605          -88.113  0.001    1192.037
Jan  1088533.654          127.311  0.003    1393.296
Feb   430992.107          597.790  0.065    1208.037

Forecast utility demand D_m (MMBtu):
Nov     715569.0
Dec    1011213.0
Jan    1265915.0
Feb    1153145.0

High-price winters H (7): 2000-01, 2002-03, 2003-04, 2010-11, 2013-14, 2018-19, 2020
-21
GPR 2026-09 = 155.9, threshold = 148.1 -> g = 1, H weights x2.0

Optimal hedge shares: Nov 0.0%, Dec 100.0%, Jan 0.0%, Feb 56.8%
Expected cost      : $13,486,120
High-price avg cost: $15,320,175  (Cmax = $15,320,175)
Shadow price of Cmax: 0.0967 $ expected / $ of Cmax

Strategy comparison:
            expected_cost  high_price_avg_cost  worst_winter_cost worst_winter  std_
cost  expected_$per_MMBtu
strategy
0% hedge       13,371,613           16,564,281         19,441,256      2002-03 3,19
9,846                3.23
25% hedge      13,478,049           15,942,228         18,250,603      2002-03 2,47
4,475                3.25
50% hedge      13,584,485           15,320,175         17,059,950      2002-03 1,75
8,608                3.28
75% hedge      13,690,922           14,698,121         15,869,297      2002-03 1,07
1,468                3.30
100% hedge     13,797,358           14,076,068         14,695,947      2014-15   53
7,634                3.33
Optimal        13,486,120           15,320,175         17,201,332      2002-03 1,93
8,258                3.25

Sensitivity - hedge shares Nov/Dec/Jan/Feb (rows theta, cols Cmax factor):
                0.90x               0.94x               0.97x               1.00x
1.03x               1.06x               1.10x             no limit
theta=1.0  infeasible  1.00/1.00/0.51/1.00  0.00/1.00/0.00/0.97  0.00/1.00/0.00/0.57
0.00/1.00/0.00/0.16  0.00/0.54/0.00/0.00  0.00/0.00/0.00/0.00  0.00/0.00/0.00/0.00
theta=1.5  infeasible  1.00/1.00/0.51/1.00  0.00/1.00/0.00/0.97  0.00/1.00/0.00/0.57
```

```
0.00/1.00/0.00/0.16  0.00/0.54/0.00/0.00  0.00/0.00/0.00/0.00  0.00/0.00/0.00/0.00
theta=2.0  infeasible  1.00/1.00/0.51/1.00  0.00/1.00/0.00/0.97  0.00/1.00/0.00/0.57
0.00/1.00/0.00/0.16  0.00/0.54/0.00/0.00  0.00/0.00/0.00/0.00  0.00/0.00/0.00/0.00
theta=3.0  infeasible  1.00/1.00/0.51/1.00  1.00/1.00/0.00/1.00  1.00/1.00/0.00/1.00
1.00/1.00/0.00/1.00  1.00/1.00/0.00/1.00  1.00/1.00/0.00/1.00  1.00/1.00/0.00/1.00
theta=4.0  infeasible  1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00
1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00  1.00/1.00/1.00/1.00

Results written to C:\Users\Shivani Murugapiran\natural-gas-hedging-optimization\out
puts\VT
```

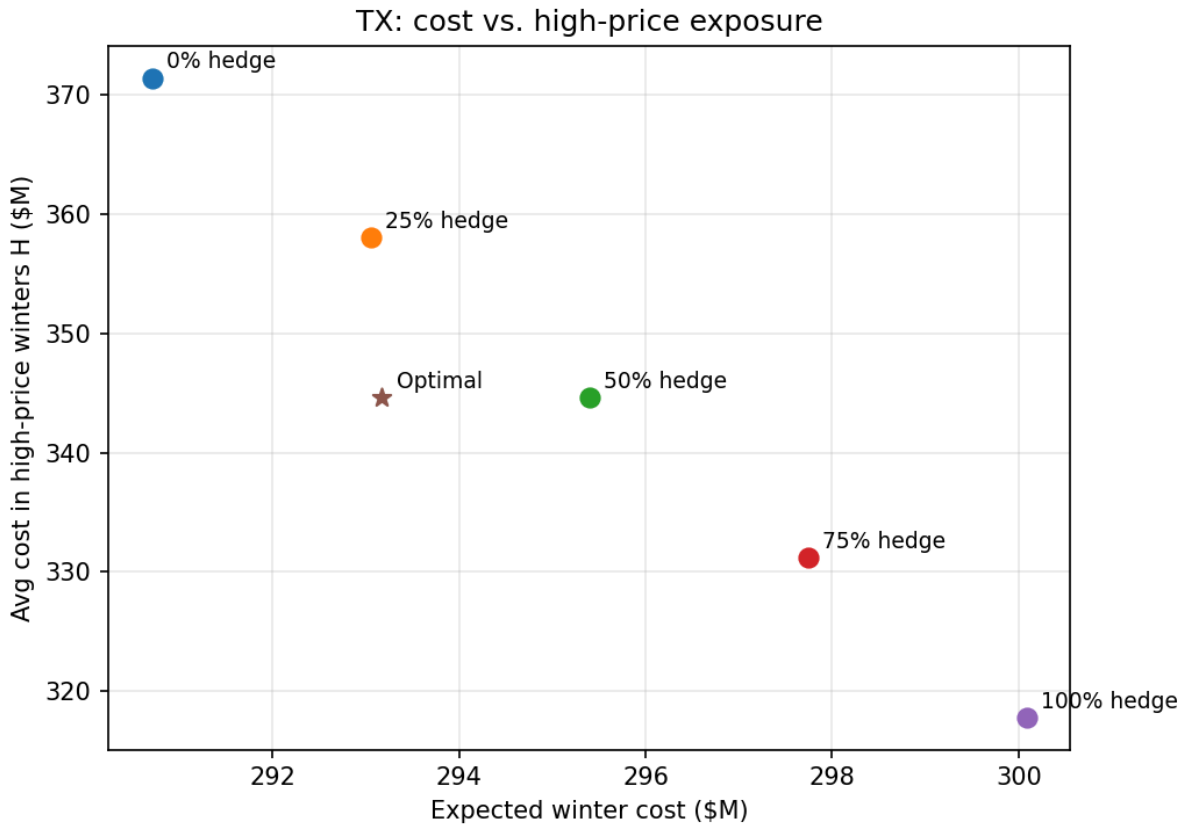
10. Results tables and charts

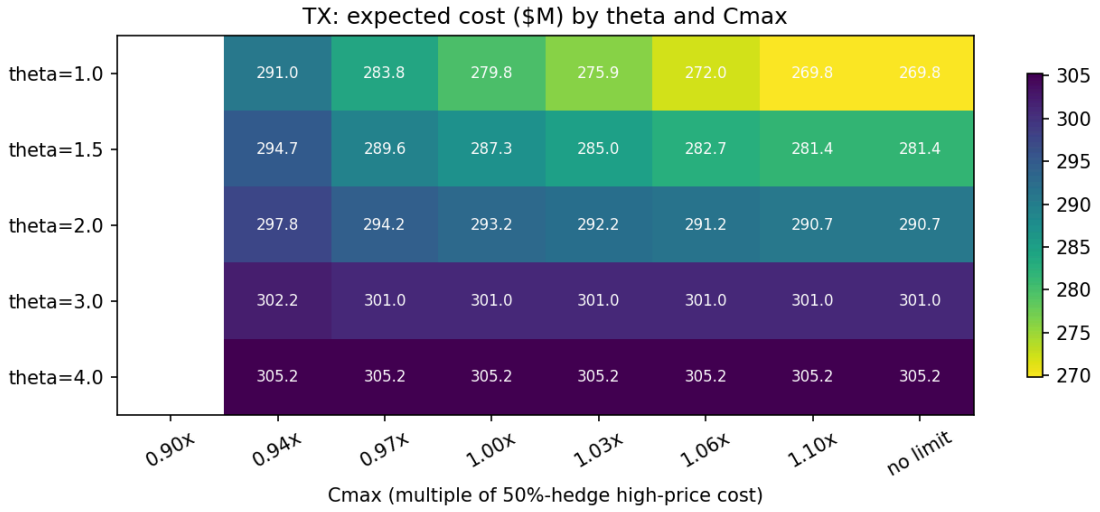
In [34]:
```
for st, (inst, sol) in results.items():
    print(f"\n=== {st} ===")
    if sol.status != "optimal":
        print("Model", sol.status)
        continue
    display(pd.DataFrame({"hedge share h_m": sol.h.round(4),
                          "hedged volume (MMBtu)": (sol.h * inst.D).round(0),
                          "forecast D_m (MMBtu)": inst.D.round(0),
                          "futures F_m": inst.F}))
    display(strategy_table(inst, sol).style.format(precision=2, thousands=","))
    for png in ["strategy_frontier.png", "sensitivity_heatmap.png"]:
        f = cfg.out_dir / st / png
        if f.exists():
            display(Image(filename=str(f)))
```

```
=== TX ===
```

	hedge share h_m	hedged volume (MMBtu)	forecast D_m (MMBtu)	futures F_m
Nov	0.0000	0.0	12828729.0	2.840
Dec	1.0000	22107291.0	22107291.0	3.233
Jan	0.0000	0.0	28736613.0	3.492
Feb	0.5746	13934281.0	24250350.0	3.427

strategy	h_Nov	h_Dec	h_Jan	h_Feb	expected_cost	high_price_avg_cost	worst_winter_cost
0% hedge	0.00	0.00	0.00	0.00	290,710,615.58	371,422,629.08	450,096,608.70
25% hedge	0.25	0.25	0.25	0.25	293,053,356.12	358,014,865.25	423,732,354.74
50% hedge	0.50	0.50	0.50	0.50	295,396,096.66	344,607,101.42	397,368,100.78
75% hedge	0.75	0.75	0.75	0.75	297,738,837.20	331,199,337.59	371,003,846.83
100% hedge	1.00	1.00	1.00	1.00	300,081,577.74	317,791,573.76	344,639,592.87
Optimal	0.00	1.00	0.00	0.57	293,176,911.51	344,607,101.42	400,399,796.68

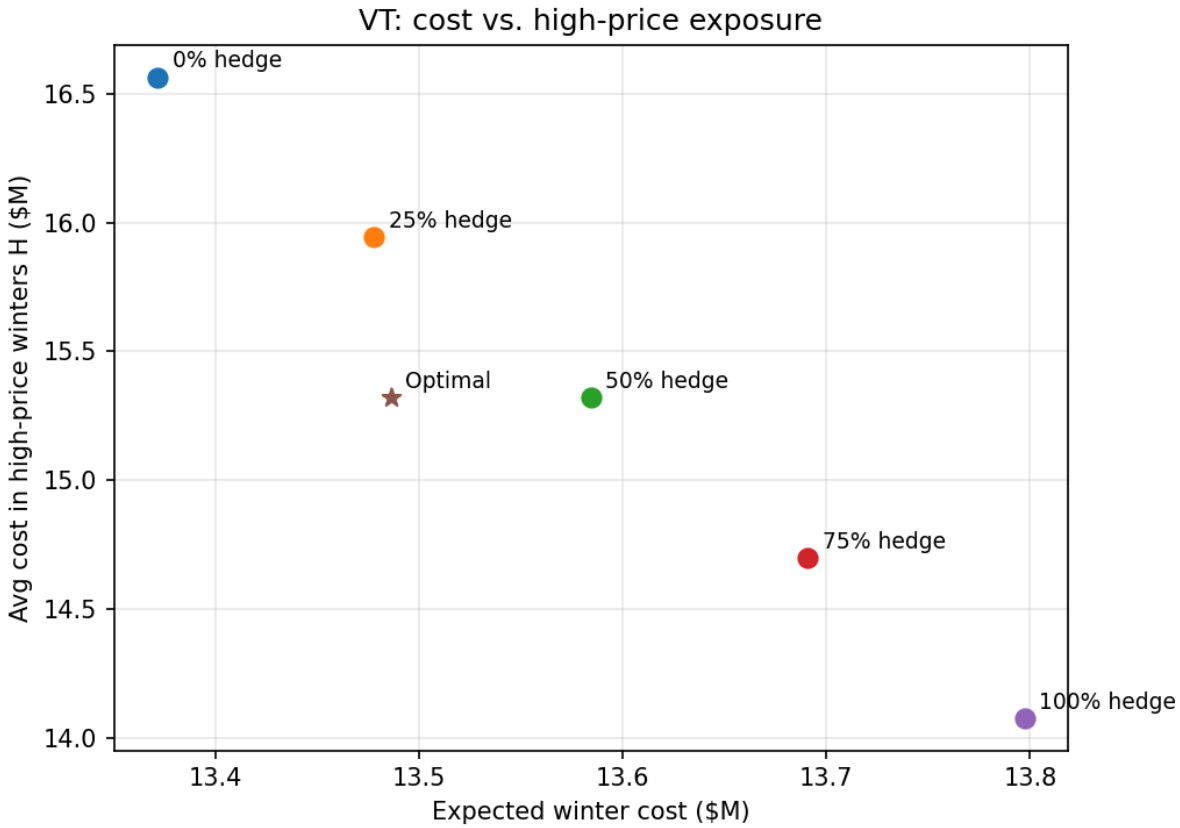


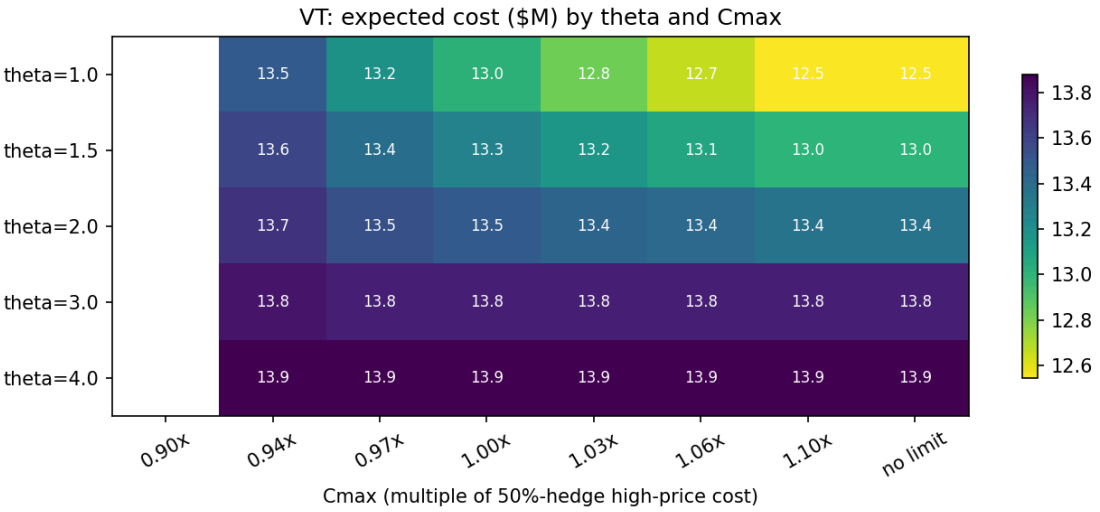


=== VT ===

	hedge share h_m	hedged volume (MMBtu)	forecast D_m (MMBtu)	futures F_m
Nov	0.0000	0.0	715569.0	2.840
Dec	1.0000	1011213.0	1011213.0	3.233
Jan	0.0000	0.0	1265915.0	3.492
Feb	0.5681	655121.0	1153145.0	3.427

strategy	h_Nov	h_Dec	h_Jan	h_Feb	expected_cost	high_price_avg_cost	worst_winter_cost
0% hedge	0.00	0.00	0.00	0.00	13,371,612.73	16,564,281.34	19,441,256.39
25% hedge	0.25	0.25	0.25	0.25	13,478,049.07	15,942,228.03	18,250,603.16
50% hedge	0.50	0.50	0.50	0.50	13,584,485.41	15,320,174.73	17,059,949.94
75% hedge	0.75	0.75	0.75	0.75	13,690,921.75	14,698,121.43	15,869,296.71
100% hedge	1.00	1.00	1.00	1.00	13,797,358.08	14,076,068.13	14,695,947.44
Optimal	0.00	1.00	0.00	0.57	13,486,119.54	15,320,174.73	17,201,331.83





In [35]:
```
# Sensitivity matrices (rows: theta, columns: Cmax as a multiple of the 50%-hedge h
for st in results:
    d = cfg.out_dir / st
    if (d / "sensitivity_hedge_shares.csv").exists():
        print(f"\n=== {st}: hedge shares Nov/Dec/Jan/Feb ===")
        display(pd.read_csv(d / "sensitivity_hedge_shares.csv", index_col=0))
        print(f"=== {st}: expected cost ($) ===")
        display(pd.read_csv(d / "sensitivity_expected_cost.csv", index_col=0)
                .style.format("{:,.0f}", na_rep="infeasible"))
```

```
=== TX: hedge shares Nov/Dec/Jan/Feb ===
```

	0.90x	0.94x	0.97x	1.00x	1.0
theta=1.0	infeasible	1.00/1.00/0.59/1.00	0.09/1.00/0.00/1.00	0.00/1.00/0.00/0.57	0.00/1.00/0.00/0
theta=1.5	infeasible	1.00/1.00/0.59/1.00	0.09/1.00/0.00/1.00	0.00/1.00/0.00/0.57	0.00/1.00/0.00/0
theta=2.0	infeasible	1.00/1.00/0.59/1.00	0.09/1.00/0.00/1.00	0.00/1.00/0.00/0.57	0.00/1.00/0.00/0
theta=3.0	infeasible	1.00/1.00/0.59/1.00	1.00/1.00/0.00/1.00	1.00/1.00/0.00/1.00	1.00/1.00/0.00/1
theta=4.0	infeasible	1.00/1.00/1.00/1.00	1.00/1.00/1.00/1.00	1.00/1.00/1.00/1.00	1.00/1.00/1.00/1

```
=== TX: expected cost ($) ===
```

	0.90x	0.94x	0.97x	1.00x	1.03x	1.06x	1.1
theta=1.0	infeasible	290,963,764	283,791,806	279,843,159	275,903,916	272,049,324	269,785,2
theta=1.5	infeasible	294,746,852	289,584,258	287,275,086	284,974,240	282,748,331	281,448,5
theta=2.0	infeasible	297,751,069	294,184,147	293,176,912	292,177,145	291,244,601	290,710,6
theta=3.0	infeasible	302,220,758	300,952,915	300,952,915	300,952,915	300,952,915	300,952,9
theta=4.0	infeasible	305,246,993	305,246,993	305,246,993	305,246,993	305,246,993	305,246,9

```
=== VT: hedge shares Nov/Dec/Jan/Feb ===
```

	0.90x	0.94x	0.97x	1.00x	1.0
theta=1.0	infeasible	1.00/1.00/0.51/1.00	0.00/1.00/0.00/0.97	0.00/1.00/0.00/0.57	0.00/1.00/0.00/0
theta=1.5	infeasible	1.00/1.00/0.51/1.00	0.00/1.00/0.00/0.97	0.00/1.00/0.00/0.57	0.00/1.00/0.00/0
theta=2.0	infeasible	1.00/1.00/0.51/1.00	0.00/1.00/0.00/0.97	0.00/1.00/0.00/0.57	0.00/1.00/0.00/0
theta=3.0	infeasible	1.00/1.00/0.51/1.00	1.00/1.00/0.00/1.00	1.00/1.00/0.00/1.00	1.00/1.00/0.00/1
theta=4.0	infeasible	1.00/1.00/1.00/1.00	1.00/1.00/1.00/1.00	1.00/1.00/1.00/1.00	1.00/1.00/1.00/1

```
=== VT: expected cost ($) ===
```

	0.90x	0.94x	0.97x	1.00x	1.03x	1.06x	1.10x	
theta=1.0	infeasible	13,485,555	13,185,750	13,010,624	12,835,497	12,663,703	12,543,884	12,
theta=1.5	infeasible	13,590,602	13,377,943	13,275,654	13,173,366	13,074,027	13,005,241	13,
theta=2.0	infeasible	13,674,022	13,530,566	13,486,120	13,441,673	13,399,873	13,371,613	13,
theta=3.0	infeasible	13,798,134	13,750,128	13,750,128	13,750,128	13,750,128	13,750,128	13,
theta=4.0	infeasible	13,878,649	13,878,649	13,878,649	13,878,649	13,878,649	13,878,649	13,

In [ ]: